D20A/ 37

EXPERIMENT 2

Aim: Create a Blockchain using Python

Theory:

Blockchain is a distributed and immutable ledger that records data in a sequence of linked blocks. Each block contains a timestamp, proof (from Proof-of-Work), and the hash of the previous block, ensuring integrity and security.

In this program:

1. **Block Creation** – A block is generated using `create_block()` with a proof and previous hash.

2. **Proof-of-Work (PoW)** – Implemented to mine a block by solving a cryptographic puzzle (finding a hash with leading zeros).

3. **Hashing** – SHA-256 algorithm ensures data immutability.

4. **Validation** – `is_chain_valid()` checks that every block correctly references the previous hash and satisfies PoW conditions.

5. **Flask Web API** – Routes `/mine_block`, `/get_chain`, and `/is_valid` allow mining, fetching the chain, and verifying blockchain integrity through a browser or API client.

This program runs on a local server and simulates mining a blockchain without a network of nodes, making it an ideal introductory model to understand blockchain basics.

Code:

```
# Install Flask before running:
# pip install flask

# Importing required libraries
import datetime
import hashlib
import json

from flask import Flask, jsonify, request
```

```
# ==============================================================
# PART 1 - BUILDING A BLOCKCHAIN
# ==============================================================

class Blockchain:

    # ----------------------------------------------------------
    # Initialize Blockchain
    # ----------------------------------------------------------
    def __init__(self):

        # Store all blocks
        self.chain = []

        # Store pending transactions
        self.transactions = []

        # Create Genesis Block
        self.create_block(
            proof=1,
            previous_hash='0',
            transactions=[]
        )

    # ----------------------------------------------------------
    # Create a Block
    # ----------------------------------------------------------
    def create_block(self, proof, previous_hash, transactions):

        block = {
            'index': len(self.chain) + 1,
            'timestamp': str(datetime.datetime.now()),
            'proof': proof,
            'previous_hash': previous_hash,
            'transactions': transactions
        }

        # Add block to blockchain
        self.chain.append(block)

        return block

    # ----------------------------------------------------------
```

```
# Create Transactions
# ----------------------------------------------------
def create_transactions(self, sender, receiver, amount):

    transaction = {
        'sender': sender,
        'receiver': receiver,
        'amount': amount
    }

    # Add transaction to pending transaction list
    self.transactions.append(transaction)

    # Return the block number in which
    # this transaction will be stored
    return len(self.chain) + 1

# ----------------------------------------------------
# Get Previous Block
# ----------------------------------------------------
def get_previous_block(self):

    return self.chain[-1]

# ----------------------------------------------------
# Proof of Work
# ----------------------------------------------------
def proof_of_work(self, previous_proof):

    new_proof = 1
    check_proof = False

    while check_proof is False:

        # Calculate hash for the current proof
        hash_operation = hashlib.sha256(
            str(
                new_proof ** 2 - previous_proof ** 2
            ).encode()
        ).hexdigest()

        # Cryptographic puzzle:
        # Hash must have 3 leading zeros
        if hash_operation[:3] == '000':
```

```
            check_proof = True

        else:

            new_proof += 1

    # Golden nonce
    return new_proof

# ---------------------------------------------------------
# Hash a Block
# ---------------------------------------------------------
def hash(self, block):

    # Convert block into JSON
    encoded_block = json.dumps(
        block,
        sort_keys=True
    ).encode()

    # Generate SHA-256 hash
    return hashlib.sha256(encoded_block).hexdigest()

# ---------------------------------------------------------
# Check Whether Blockchain is Valid
# ---------------------------------------------------------
def is_chain_valid(self, chain):

    # Start with Genesis Block
    previous_block = chain[0]

    block_index = 1

    while block_index < len(chain):

        block = chain[block_index]

        # Check whether previous hash is correct
        if block['previous_hash'] != self.hash(previous_block):

            return False

        # Get proofs
```

```
            previous_proof = previous_block['proof']
            proof = block['proof']

            # Calculate Proof-of-Work hash
            hash_operation = hashlib.sha256(
                str(
                    proof ** 2 - previous_proof ** 2
                ).encode()
            ).hexdigest()

            # Hash must have 3 leading zeros
            if hash_operation[:3] != '000':

                return False

            previous_block = block

            block_index += 1

        return True


# ============================================================
# PART 2 - FLASK WEB APPLICATION
# ============================================================

# Create Flask application
app = Flask(__name__)

# Create Blockchain object
blockchain = Blockchain()


# ============================================================
# ADD TRANSACTION
# ============================================================

@app.route('/add_transaction', methods=['POST'])
def add_transaction():

    # Get JSON data from Postman
    json_data = request.get_json()

    # Get transaction details
```

```
    sender = json_data['sender']
    receiver = json_data['receiver']
    amount = json_data['amount']

    # Create transaction
    block_index = blockchain.create_transactions(
        sender,
        receiver,
        amount
    )

    # Prepare response
    response = {
        'message': 'Transaction added successfully.',
        'sender': sender,
        'receiver': receiver,
        'amount': amount,
        'block_index': block_index
    }

    return jsonify(response), 201


# ============================================================
# MINE A NEW BLOCK
# ============================================================

@app.route('/mine_block', methods=['GET'])
def mine_block():

    # ------------------------------------------------------
    # Requirement:
    # Mine only when transaction list is not empty
    # ------------------------------------------------------

    if len(blockchain.transactions) == 0:

        return jsonify({
            'message': 'No transactions available. Add a transaction before mining.'
        }), 400

    # ------------------------------------------------------
    # Copy the pending transactions
    # ------------------------------------------------------
```

```
transactions_to_mine = blockchain.transactions.copy()

# ------------------------------------------------------
# Remove transactions from pending list BEFORE mining
# ------------------------------------------------------

blockchain.transactions = []

# ------------------------------------------------------
# Get previous block
# ------------------------------------------------------

previous_block = blockchain.get_previous_block()

previous_proof = previous_block['proof']

# ------------------------------------------------------
# Find Golden Nonce using Proof of Work
# ------------------------------------------------------

proof = blockchain.proof_of_work(previous_proof)

# ------------------------------------------------------
# Get hash of previous block
# ------------------------------------------------------

previous_hash = blockchain.hash(previous_block)

# ------------------------------------------------------
# Create new block
# ------------------------------------------------------

block = blockchain.create_block(
    proof,
    previous_hash,
    transactions_to_mine
)

# ------------------------------------------------------
# Prepare response
# ------------------------------------------------------

response = {
```

```
        'message': 'Congratulations, you just mined a block!',
        'index': block['index'],
        'timestamp': block['timestamp'],
        'proof': block['proof'],
        'previous_hash': block['previous_hash'],
        'transactions': block['transactions']
    }

    return jsonify(response), 200


# ============================================================
# GET THE FULL BLOCKCHAIN
# ============================================================

@app.route('/get_chain', methods=['GET'])
def get_chain():

    response = {
        'chain': blockchain.chain,
        'length': len(blockchain.chain)
    }

    return jsonify(response), 200


# ============================================================
# CHECK WHETHER BLOCKCHAIN IS VALID
# ============================================================

@app.route('/is_valid', methods=['GET'])
def is_valid():

    valid = blockchain.is_chain_valid(
        blockchain.chain
    )

    if valid:

        response = {
            'message': 'All good. The Blockchain is valid.'
        }

    else:
```

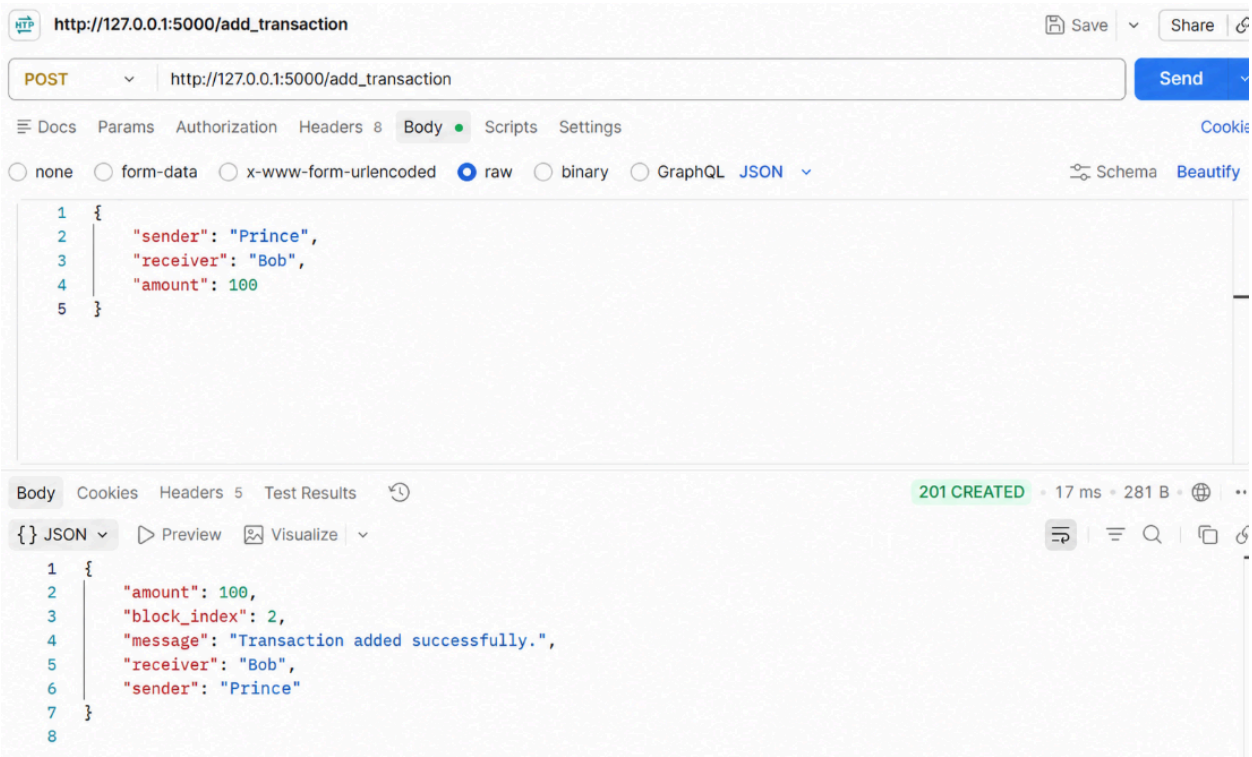
```
        response = {
            'message': 'Houston, we have a problem. The Blockchain is not valid.'
        }

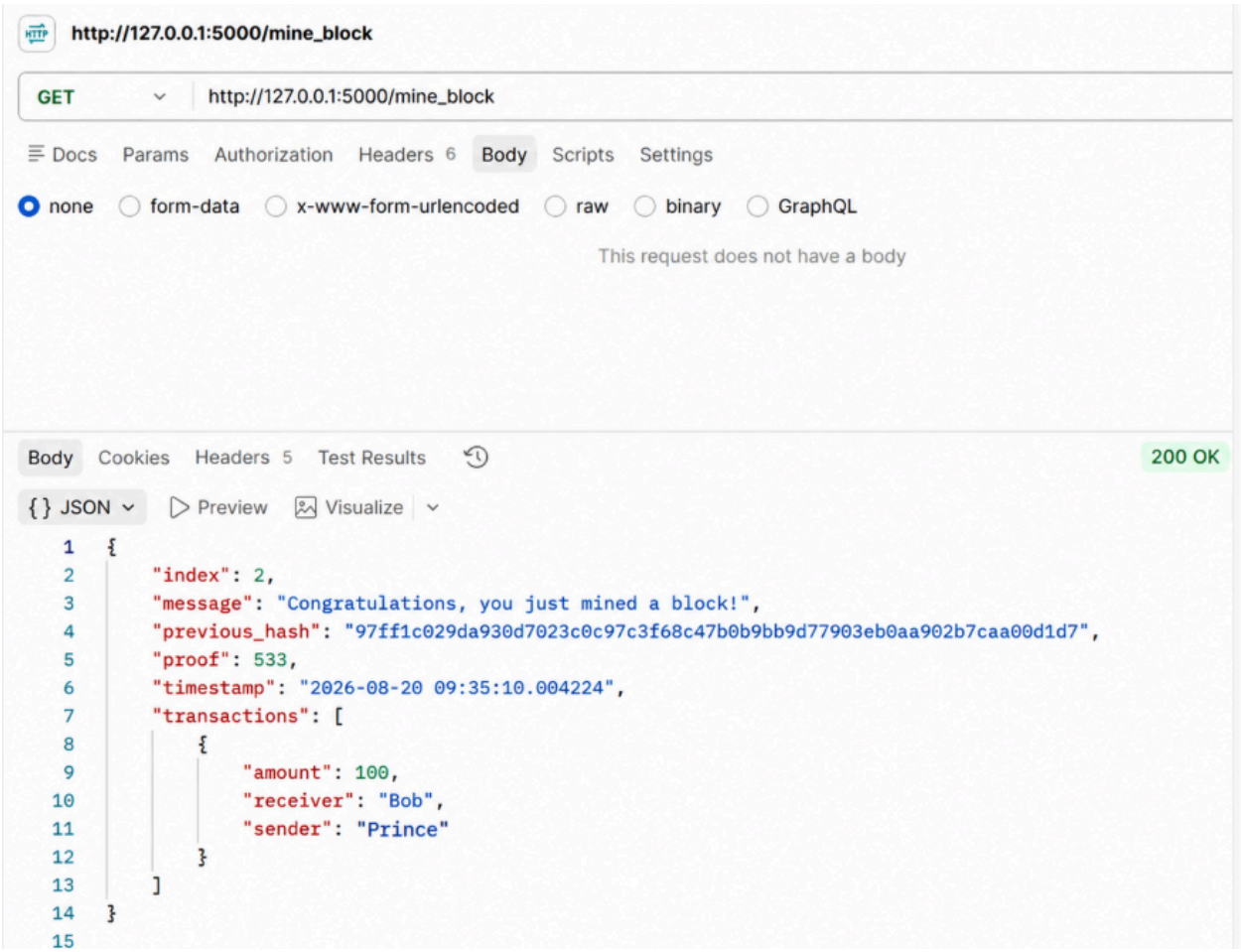
    return jsonify(response), 200

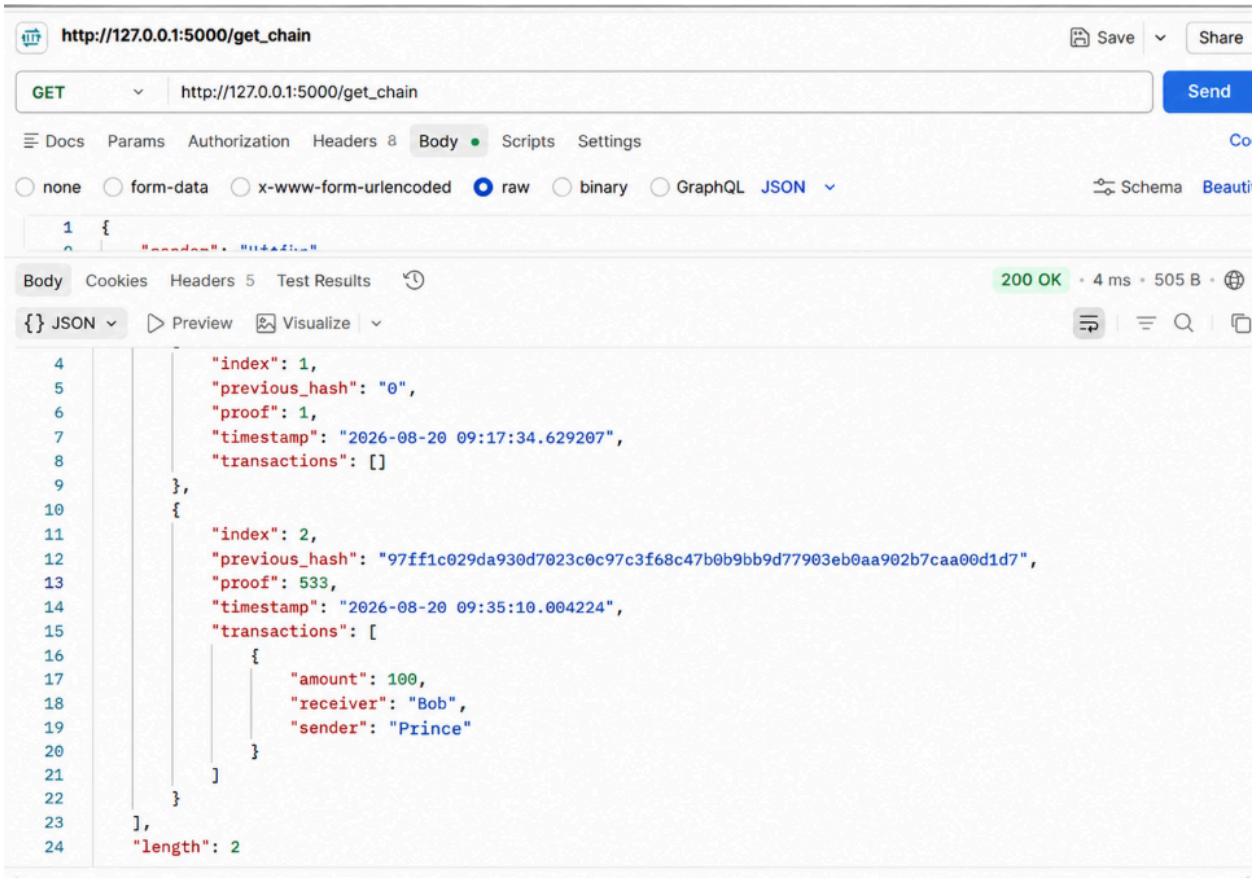

# ============================================================
# RUN FLASK APPLICATION
# ============================================================

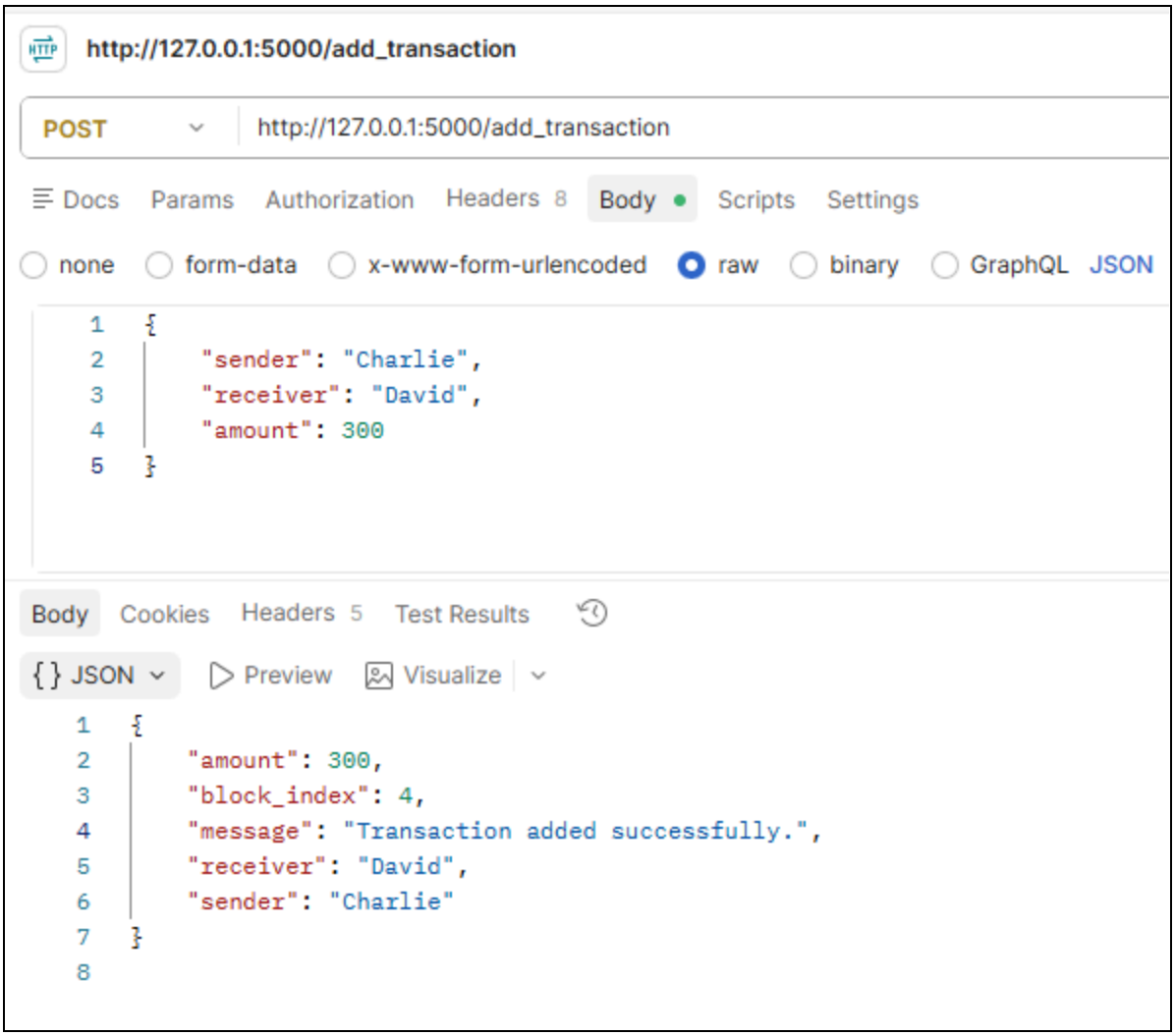
app.run(
    host='0.0.0.0',
    port=5000
)
```

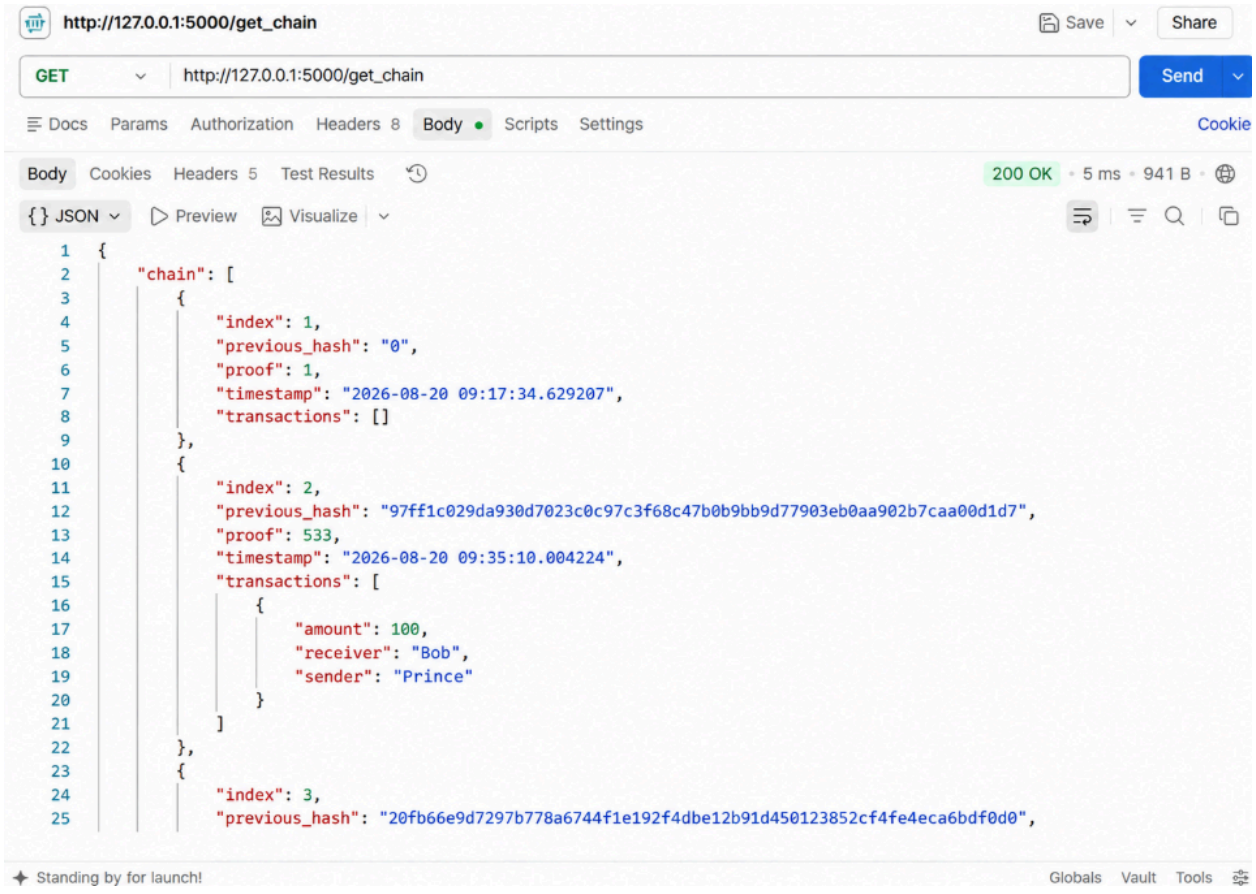
Output:




http://127.0.0.1:5000/mine_block
GET
http://127.0.0.1:5000/mine_block
Docs
Params
Authorization
Headers 6
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
This request does not have a body
Body
Cookies
Headers 5
Test Results
200 OK
{} JSON
Preview
Visualize
{
"index": 2,
"message": "Congratulations, you just mined a block!",
"previous_hash": "97ff1c029da930d7023c0c97c3f68c47b0b9bb9d77903eb0aa902b7caa00d1d7",
"proof": 533,
"timestamp": "2026-08-20 09:35:10.004224",
"transactions": [
{
"amount": 100,
"receiver": "Bob",
"sender": "Prince"
}
]
}


http://127.0.0.1:5000/get_chain
Save
Share
GET
http://127.0.0.1:5000/get_chain
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
Schema
Body
Cookies
Headers 5
Test Results
200 OK
4 ms
505 B
JSON
Preview
Visualize
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-20 09:17:34.629207",
"transactions": []
"index": 2,
"previous_hash": "97ff1c029da930d7023c0c97c3f68c47b0b9bb9d77903eb0aa902b7caa00d1d7",
"proof": 533,
"timestamp": "2026-08-20 09:35:10.004224",
"transactions": [
"amount": 100,
"receiver": "Bob",
"sender": "Prince"
"length": 2


http://127.0.0.1:5000/add_transaction
POST
http://127.0.0.1:5000/add_transaction
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
none
form-data
x-www-form-urlencoded
raw
binary
GraphQL
JSON
{
"sender": "Charlie",
"receiver": "David",
"amount": 300
}
Body
Cookies
Headers 5
Test Results
{ } JSON
Preview
Visualize
{
"amount": 300,
"block_index": 4,
"message": "Transaction added successfully.",
"receiver": "David",
"sender": "Charlie"
}


http://127.0.0.1:5000/get_chain
Save
Share
GET
http://127.0.0.1:5000/get_chain
Send
Docs
Params
Authorization
Headers 8
Body
Scripts
Settings
Cookie
Body
Cookies
Headers 5
Test Results
200 OK
5 ms
941 B
{} JSON
Preview
Visualize
{
"chain": [
{
"index": 1,
"previous_hash": "0",
"proof": 1,
"timestamp": "2026-08-20 09:17:34.629207",
"transactions": []
},
{
"index": 2,
"previous_hash": "97ff1c029da930d7023c0c97c3f68c47b0b9bb9d77903eb0aa902b7caa00d1d7",
"proof": 533,
"timestamp": "2026-08-20 09:35:10.004224",
"transactions": [
{
"amount": 100,
"receiver": "Bob",
"sender": "Prince"
}
]
},
{
"index": 3,
"previous_hash": "20fb66e9d7297b778a6744f1e192f4dbe12b91d450123852cf4fe4eca6bdf0d0",
Standing by for launch!
Globals
Vault
Tools

http://127.0.0.1:5000/get_chain

Save

GET http://127.0.0.1:5000/get_chain

Docs Params Authorization Headers 8 Body Scripts Settings

Body Cookies Headers 5 Test Results 200 OK 5 ms

{ } JSON Preview Visualize


```
            "transactions": [
                {
                    "amount": 250,
                    "receiver": "Charlie",
                    "sender": "Bob"
                }
            ]
        },
        {
            "index": 4,
            "previous_hash": "1885af888914cb87175ad2b89897c7c351b885ed09b79e30891d4cafa96a076f",
            "proof": 396,
            "timestamp": "2026-08-20 09:47:35.136716",
            "transactions": [
                {
                    "amount": 300,
                    "receiver": "David",
                    "sender": "Charlie"
                }
            ]
        }
    ],
    "length": 4
}
```


http://127.0.0.1:5000/is_valid

GET http://127.0.0.1:5000/is_valid

Docs Params Authorization Headers 8 Body Scripts Settings

Body Cookies Headers 5 Test Results

{ } JSON Preview Visualize


```
{
    "message": "All good. The Blockchain is valid."
}
```

```
127.0.0.1 - - [20/Aug/2026 09:35:10] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 09:37:10] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 09:40:44] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [20/Aug/2026 09:40:54] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 09:46:09] "GET /mine_block HTTP/1.1" 400 -
127.0.0.1 - - [20/Aug/2026 09:46:57] "POST /add_transaction HTTP/1.1" 201 -
127.0.0.1 - - [20/Aug/2026 09:47:35] "GET /mine_block HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 09:47:59] "GET /get_chain HTTP/1.1" 200 -
127.0.0.1 - - [20/Aug/2026 09:49:07] "GET /is_valid HTTP/1.1" 200 -
```

Conclusion:

We successfully created a basic blockchain using Python and Flask that supports block mining, chain retrieval, and validity checks. Proof-of-Work ensures security by making block creation computationally intensive, while cryptographic hashing guarantees immutability. The implementation demonstrates core blockchain principles—decentralization, immutability, and consensus—in a simplified environment, providing a strong foundation for building more advanced blockchain systems.